

Way Back Home - Building an ADK Bi-Directional Streaming Agent

來源：<https://codelabs.developers.google.com/way-back-home-level-3/instructions?hl=zh-tw>

步驟數：6

在本程式碼研究室中，您將使用 Agent 建構全端即時多模態 AI 應用程式。您將開發 React 前端，擷取即時視訊和音訊，並使用 Google 的 ADK 開發 Python 後端，處理這些資料。建立雙向 WebSocket 連線後，您會將媒體串流至 Gemini Live API，該 API 會解讀手勢來執行特定指令，這個程序稱為「工具呼叫」。

目錄

1. [1. 使命](#)
2. [2. 設定環境](#)
3. [3. 校準通訊連結 \(WebSocket\)](#)
4. [4. 多模態代理程式](#)
5. [5. 雙向串流流程](#)
6. [6. 部署至正式環境 \(選用\)](#)

1. 使命



你漂流在未知的星區，周遭一片寂靜。巨大的 ****太陽脈衝**** 撕裂了你的船隻，讓你困在星圖上不存在的宇宙空間。

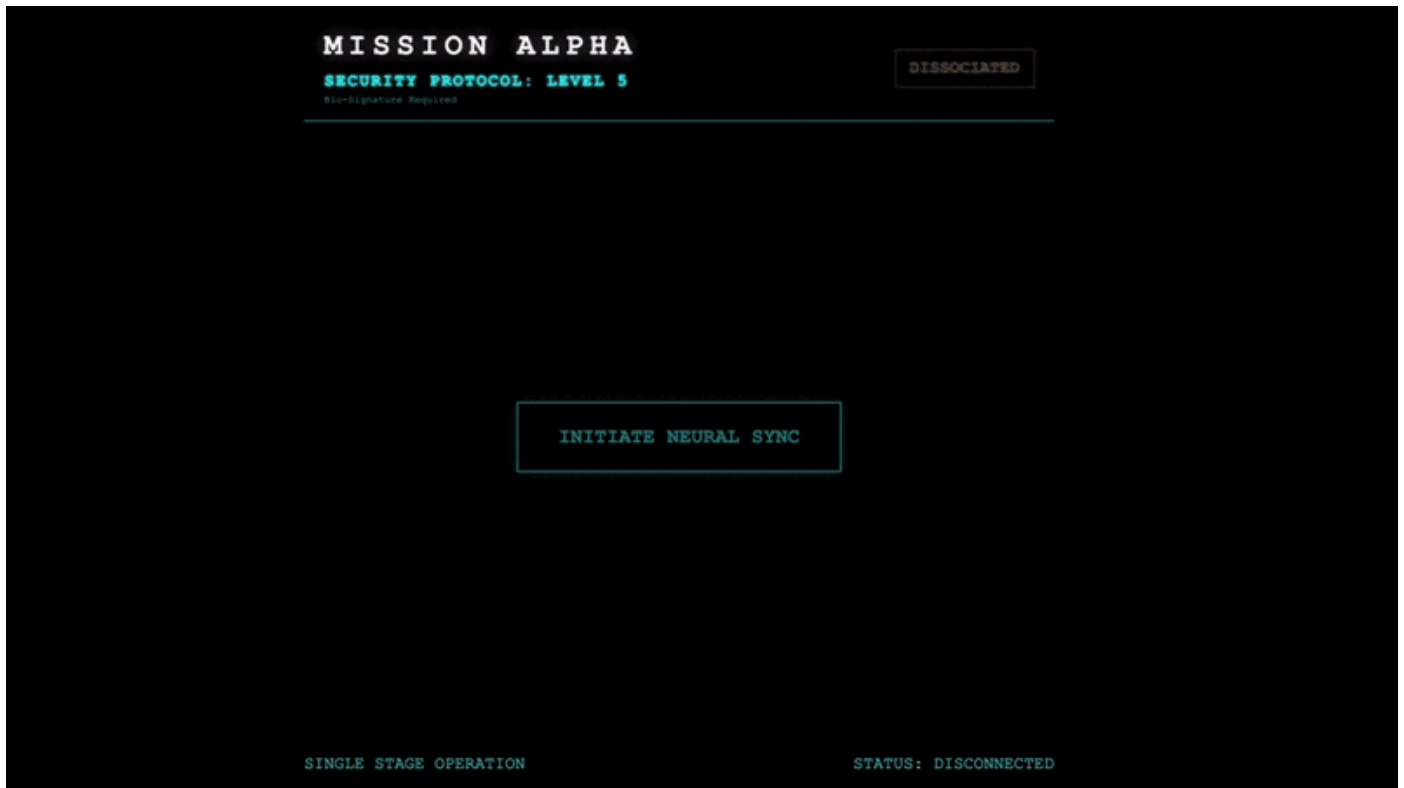
經過多日艱辛的維修，你終於感受到腳下引擎的嗡嗡聲。火箭已修復，你甚至還設法確保了與母艦的遠距上行連結。可以出發。你已準備好回家。但正當你準備啟動跳躍引擎時，一陣靜電干擾中傳來求救訊號。感應器在「峽谷」中偵測到五個微弱的熱訊號，這個區域地形崎嶇，重力扭曲，主艦無法進入。他們也是探險家，與你一樣經歷了那場幾乎奪走性命的風暴。你無法留下這些裝置。

你轉向 **Alpha-Drone Rescue Scout**，這艘靈活的小船是唯一能通過狹窄峽谷的船隻，但問題是，太陽脈衝對核心邏輯執行了「系統重設」。Scout 的控制系統沒有反應。雖然已啟動電源，但機載電腦是空白狀態，無法處理手動飛行員指令或飛行路徑。

挑戰

如要拯救倖存者，你必須完全繞過偵察兵的損壞電路。您還有一個不得已的選擇：**建立 AI 代理程式，建立生物特徵辨識神經同步**。這個代理程式會做為即時橋梁，讓您透過自己的生物輸入內容手動控制 Rescue Scout。你不會使用搖桿或鍵盤，而是直接將意圖連線到太空船的導航網路。

如要鎖定連結，請在 Scout 的光學感應器前方執行**同步處理通訊協定**。AI 代理必須透過精確的即時交握，辨識您的生物特徵。

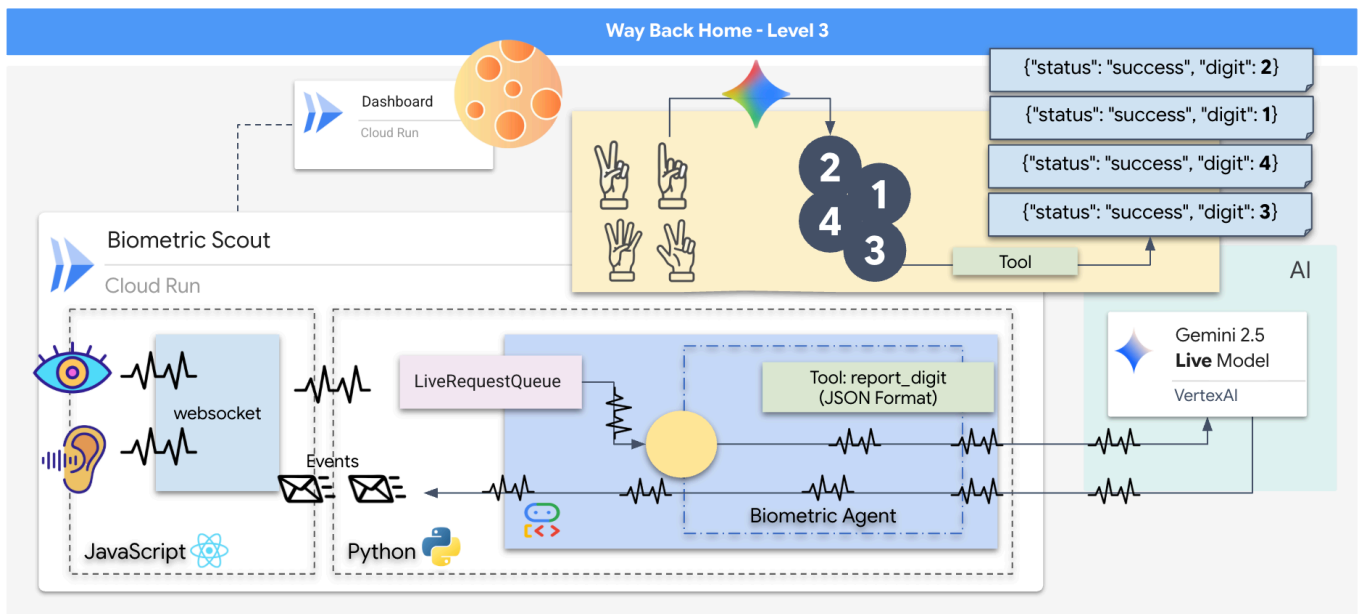


您的任務目標：

1. 刻印神經元核心：定義可辨識多模態輸入內容的 ADK 代理程式。
2. 建立連線：建構雙向 WebSocket 管道，將視覺資料從 Scout 串流至 AI。
3. 啟動握手程序：站在感應器前方，依序伸出 1 到 5 根手指。

如果成功，「生物特徵辨識同步」就會啟動。AI 會鎖定神經連結，讓您完全手動控制，發射偵察兵並帶領倖存者回家。

建構目標



您將建構「Biometric Neural Sync」應用程式，這是一套即時 AI 輔助系統，可做為救援無人機的控制介面。這個系統包含：

- **React 前端**：船艦的「駕駛艙」，可擷取網路攝影機的即時影像和麥克風的音訊。

- **Python 後端**：使用 FastAPI 建構的高效能伺服器，並使用 Google 的 Agent Development Kit (ADK) 管理 LLM 的邏輯和狀態。
- **多模態 AI 代理程式**：作業的「大腦」，透過 `google-genai` SDK 使用 Gemini Live API，同時處理及瞭解影片和音訊串流。
- **雙向 WebSocket 管道**：這個「神經系統」會在前端和 AI 之間建立持續的低延遲連線，實現即時互動。

學習目標

技術 / 概念	說明
後端 AI 代理	使用 Python 和 FastAPI 建構有狀態的 AI 代理。使用 Google 的 ADK (Agent Development Kit) 管理指令和記憶體，並使用 <code>google-genai</code> SDK 與 Gemini 模型互動。
前端 UI	使用 React 開發動態使用者介面，直接從瀏覽器擷取及串流即時影像和音訊。
即時通訊	實作 WebSocket 管道，進行全雙工低延遲通訊，讓使用者和 AI 同時互動。
多模態 AI	運用 Gemini Live API 處理及解讀同步影片和音訊串流，讓 AI 同時「看見」和「聽見」。
工具呼叫	啟用 AI，在收到視覺觸發條件時執行特定 Python 函式，縮小模型智慧與現實世界行動之間的差距。
全端部署	使用 Docker 將整個應用程式 (React 前端和 Python 後端) 容器化，並以可擴充的無伺服器服務形式部署至 Google Cloud Run 。

步驟 2 · 約 0 分鐘

2. 設定環境

info

從做中學 免付費

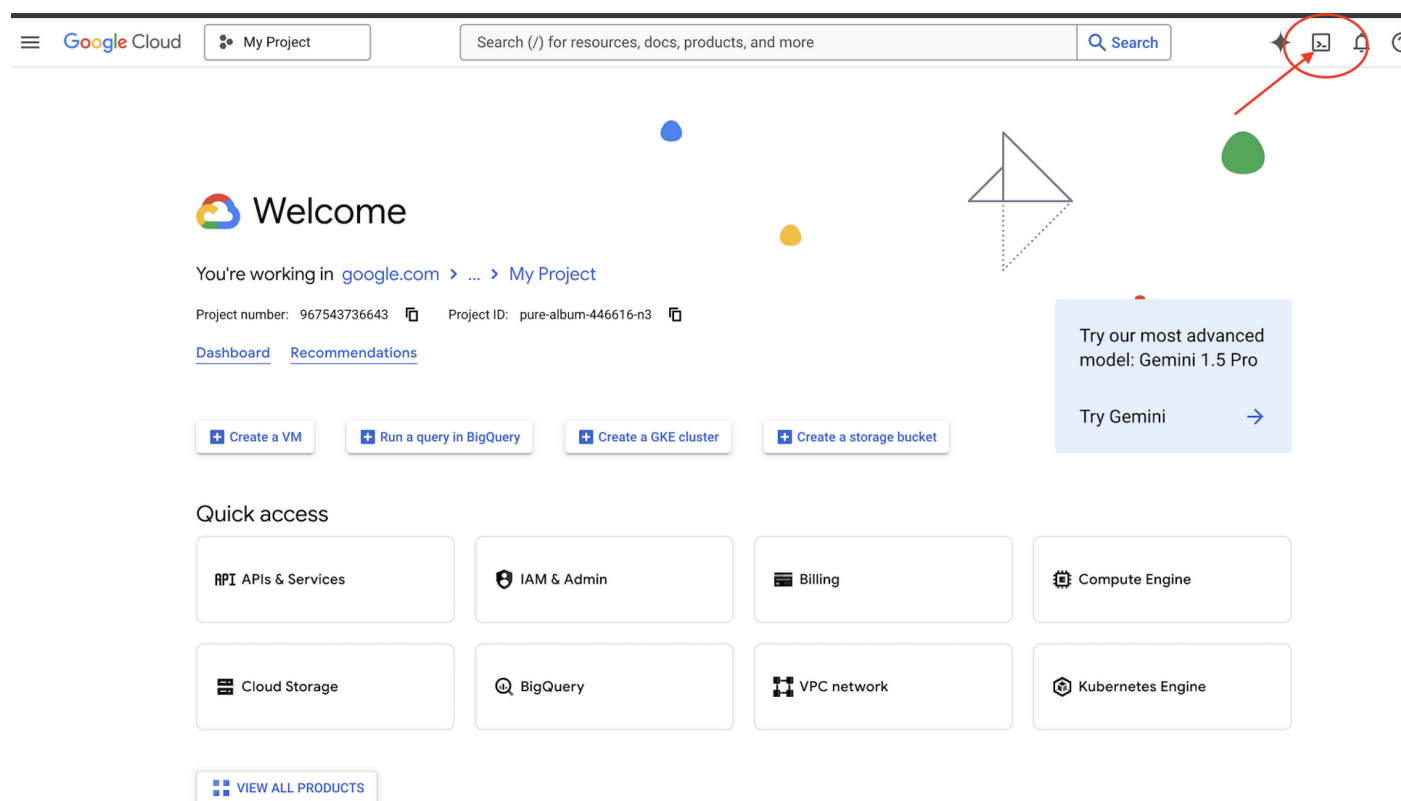
立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

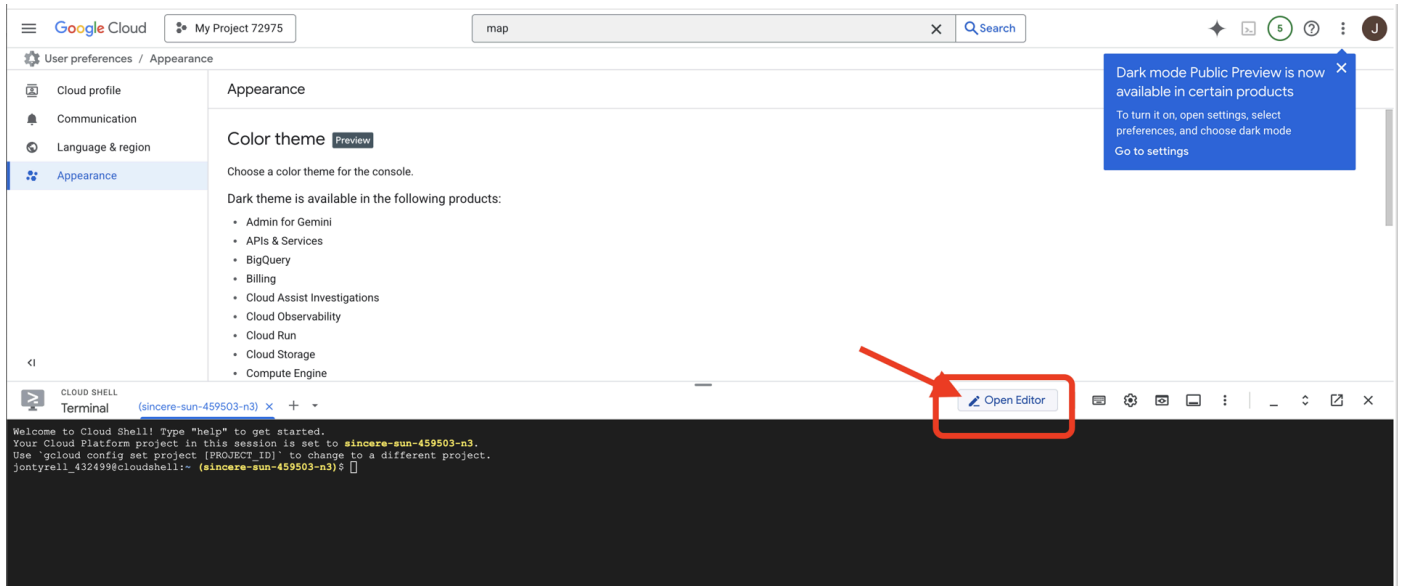
存取 Cloud Shell

首先，我們要開啟 Cloud Shell。這是以瀏覽器為基礎的終端機，已預先安裝 Google Cloud SDK 和其他必要工具。

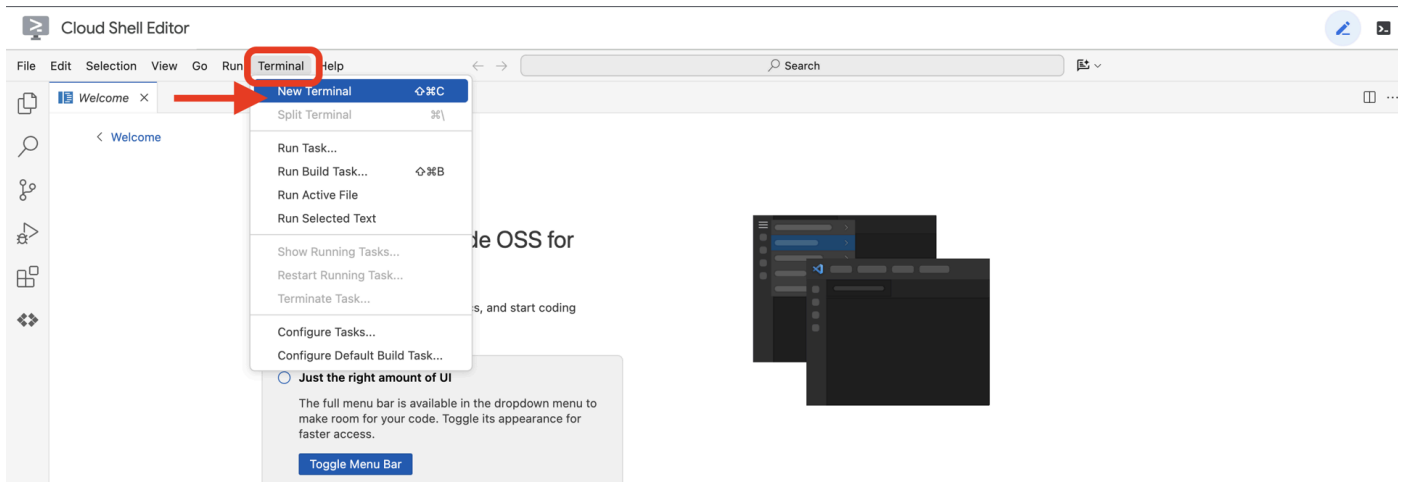
👉 點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」(這是 Cloud Shell 窗格頂端的終端機形狀圖示)



👉 按一下「Open Editor」按鈕 (類似於有鉛筆的開啟資料夾)。視窗中會開啟 Cloud Shell 程式碼編輯器。左側會顯示檔案總管。



👉 在雲端 IDE 中開啟終端機，



👉 🖥️ 在終端機中，使用下列指令驗證您是否已通過驗證，以及專案是否已設為您的專案 ID：

```
gcloud auth list
```

您的帳戶應該會顯示為 (ACTIVE)。

必要條件

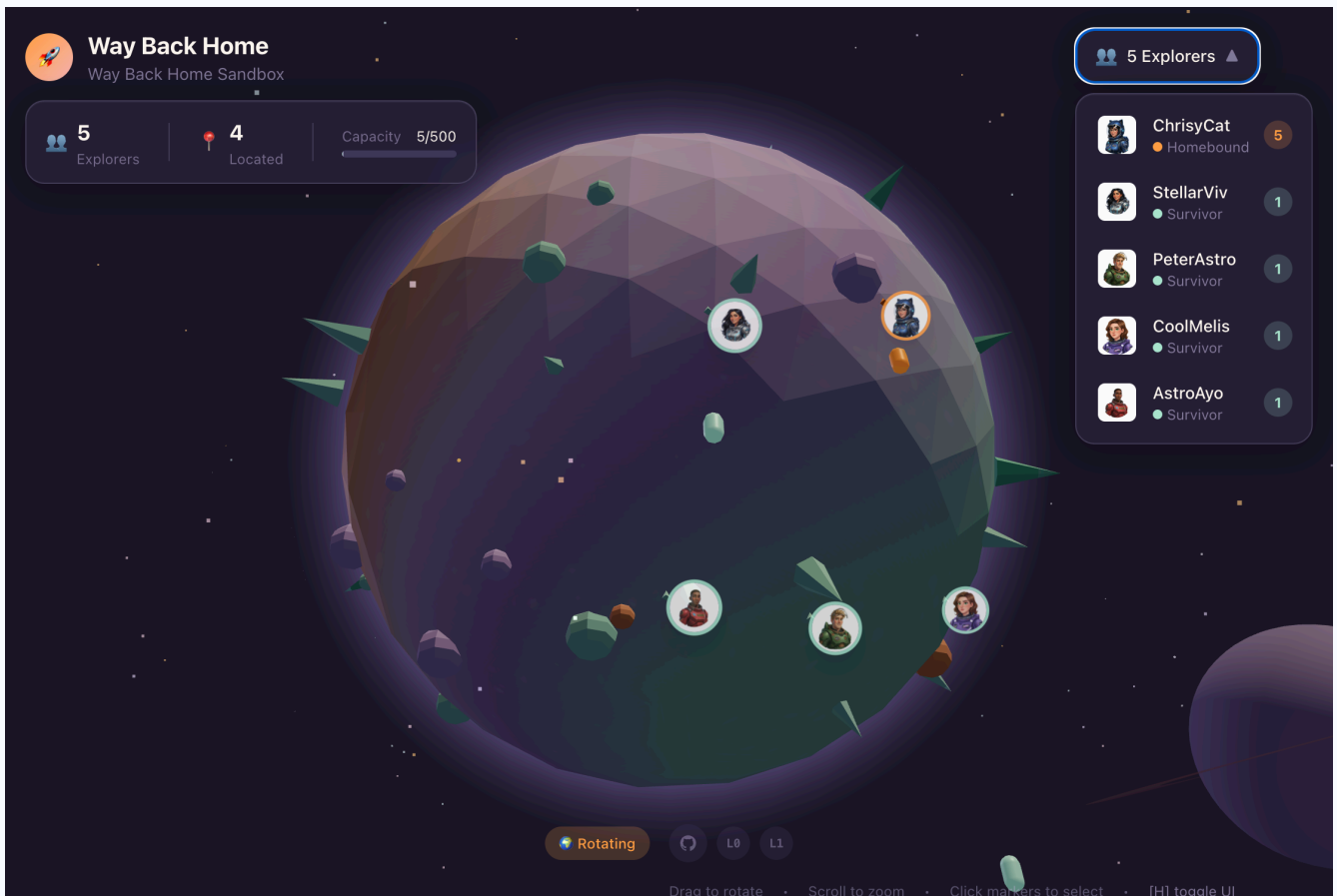
第 0 級為選用 (但建議使用)

您可以在 0 級完成這項任務，但建議先完成 0 級，享受更身歷其境的體驗，在完成任務的過程中，看著信號在世界地圖上亮起。

i 如果你已完成第 0 級，請驗證：

[按這裡開始第 0 級](#)

- [] `config.json` 在專案根目錄中，並提供參與者 ID 和座標
- [] 你的虛擬人偶會顯示在世界地圖上
- [] 座標處顯示 (昏暗) 信標
- [] 您已在工作目錄中複製 `way-back-home` 資料夾



執行準備指令碼，將工作區切換至新的任務參數。

```
cd $HOME/way-back-home/level_3
chmod +x scripts/level_0_complete_prep.sh
./scripts/level_0_complete_prep.sh
```

指令碼會嘗試偵測有效的 Google Cloud 設定。

- 重複使用第 0 級專案：如果偵測到的專案 ID 正確無誤，只要按下 **Enter** 鍵即可。
- 切換專案：輸入要使用的新專案 ID，然後按下 **Enter** 鍵。

i 如果您略過第 0 級，則需要佈建新的環境：

請按照這些步驟設定必要基礎架構和 Google Cloud 專案。

複製並初始化

在終端機中執行下列指令，下載任務檔案並啟動設定指令碼：

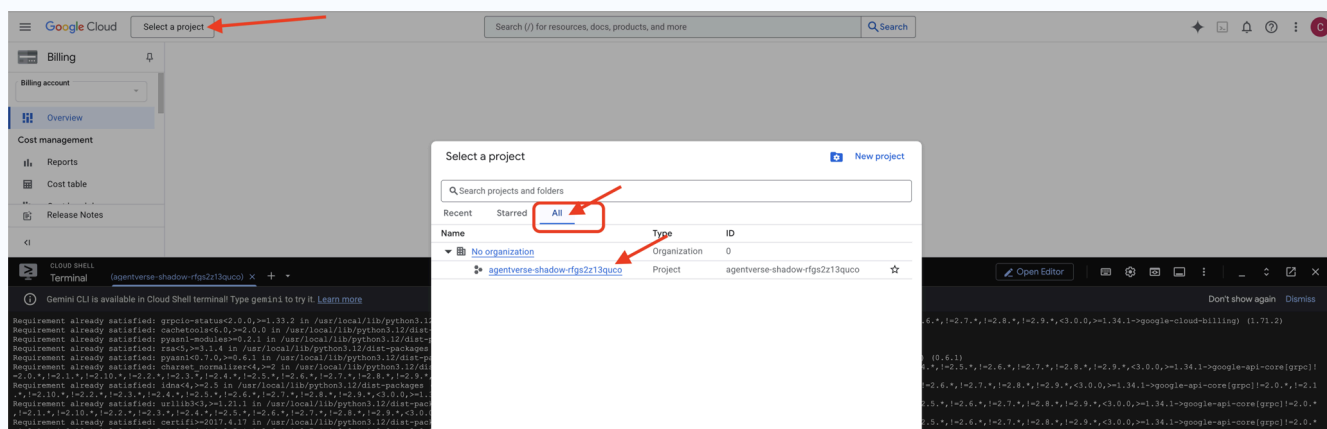
```
git clone https://github.com/gca-americas/way-back-home.git
cd $HOME/way-back-home/level_3/scripts/
chmod +x init.sh
. init.sh
```

指令碼會處理 Google Cloud 專案的建立作業和基本設定。

確認專案選項

重要步驟：即使指令碼執行完畢，Google Cloud 控制台可能仍會查看舊專案或預設專案。你必須手動切換：

1. 前往 console.cloud.google.com。
2. 按一下頁面頂端的「專案選取器」下拉式選單。
3. 按一下「全部」分頁標籤 (新專案可能尚未顯示在「最近」中)。
4. 選取指令碼剛建立的「專案 ID」。



設定專案環境

返回終端機，設定有效專案並啟用必要的 Google Cloud 服務 (Cloud Run、Vertex AI 等)，完成設定。

在終端機中設定專案 ID：

```
gcloud config set project $(cat ~/project_id.txt) --quiet
```

啟用必要服務：

```
gcloud services enable compute.googleapis.com \
  artifactregistry.googleapis.com \
  run.googleapis.com \
  cloudbuild.googleapis.com \
  iam.googleapis.com \
  aiplatform.googleapis.com
```

安裝依附元件

👉🏠 前往 Level 並安裝必要的 Python 套件：

```
cd $HOME/way-back-home/level_3
uv sync
```

主要依附元件如下：

套件	目的
<code>fastapi</code>	高效能網頁架構，適用於衛星電台和 SSE 串流
<code>uvicorn</code>	執行 FastAPI 應用程式所需的 ASGI 伺服器
<code>google-adk</code>	用於建構 Formation Agent 的 Agent Development Kit
<code>google-genai</code>	用來存取 Gemini 模型的原生用戶端
<code>websockets</code>	支援即時雙向通訊
<code>python-dotenv</code>	管理環境變數和設定密鑰

驗證設定

開始撰寫程式碼前，請先確認所有系統都正常運作。執行驗證指令碼，稽核 Google Cloud 專案、API 和 Python 依附元件。

👉🏠 執行驗證指令碼：

```
cd $HOME/way-back-home/level_3/scripts
chmod +x verify_setup.sh
. verify_setup.sh
```

👁️ 你應該會看到一連串的綠色勾號 (✅)。

- 如果看到紅十字 (❌)，請按照輸出內容中的建議修正指令 (例如 `gcloud services enable ...` 或 `pip install ...`) 操作。

- 附註：目前 `.env` 的黃色警告訊息可以接受，我們會在下一個步驟中建立該檔案。

 Verifying Mission Alpha (Level 3) Infrastructure...

✓ Google Cloud Project: xxxxxx

✓ Cloud APIs: Active

✓ Python Environment: Ready

 SYSTEMS ONLINE. READY FOR MISSION.

3. 校準通訊連結 (WebSocket)

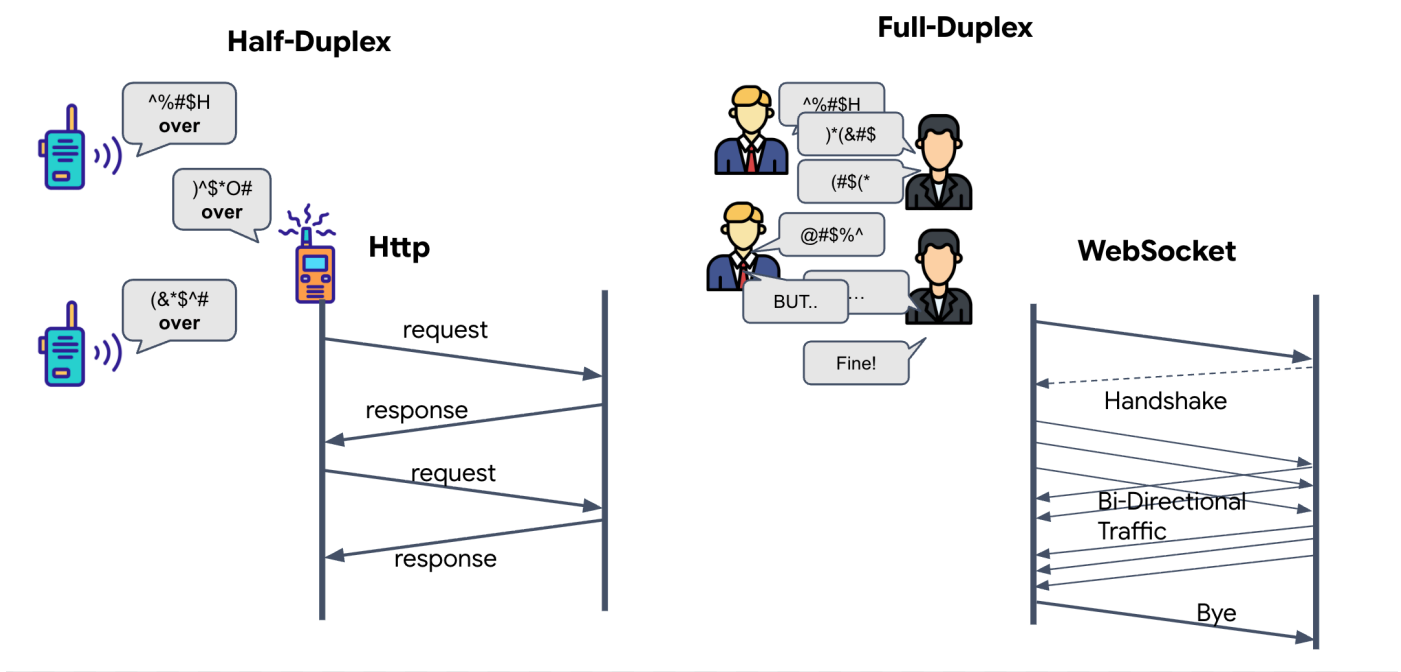
如要開始進行生物神經同步，我們需要更新船艦的內部系統。我們的首要目標是從駕駛艙擷取高保真度的影片和音訊串流。這項串流提供神經連結的基本元件：手指序列的視覺辨識和語音的音頻。

全雙工與半雙工

如要瞭解為何需要這項資訊才能進行神經元同步，請先瞭解資料流程：

- **半雙工 (標準 HTTP)**：類似對講機，一人說話，說完後說「Over」，然後換另一個人說話。你無法同時聆聽和說話。
- **全雙工 (WebSocket)**：就像面對面交談一樣，資料會同時雙向流動。瀏覽器將影片影格和音訊樣本上傳至 AI 時，AI 會同時將語音回覆和工具指令下傳至瀏覽器。

Gemini Live 需要全雙工的原因：Gemini Live API 的設計宗旨是「中斷」。假設你正在展示手指序列，但 AI 發現你做錯了，在標準 HTTP 設定中，AI 必須等待您傳送完資料，才能告知您停止傳送。透過 WebSocket，AI 可以在第 1 個影格中發現錯誤，並在您移動手部拍攝第 2 個影格時，將「中斷」訊號傳送至駕駛艙。



什麼是 WebSocket ？

在標準銀河傳輸 (HTTP) 中，您會傳送要求並等待回覆，就像寄送明信片一樣。**神經網路同步**的運作速度太慢，無法使用明信片。我們需要「帶電線」。

WebSocket 最初是標準網頁要求 (HTTP)，但隨後會「升級」為其他要求。

1. **要求**：瀏覽器會向伺服器傳送標準 HTTP 要求，並附上特殊標頭：`Upgrade: websocket`。這句話的意思是「我不想再寄明信片了，我想開始進行電話通話」。
2. **回應**：如果 AI 代理程式 (伺服器) 支援這項功能，就會傳回 `HTTP 101 Switching Protocols` 回應。
3. **轉換**：此時，HTTP 連線會由 WebSocket 通訊協定取代，但底層的 TCP/IP Socket 會保持開啟。通訊規則會立即從「要求/回應」變更為「全雙工串流」。

實作 WebSocket Hook

讓我們檢查端子台，瞭解資料流動方式。

👁️ 開啟 `$HOME/way-back-home/level_3/frontend/src/useGeminiSocket.js` 。您會看到已設定的標準 WebSocket 生命週期事件處理常式。以下是通訊系統的架構：

```
const connect = useCallback(() => {
  if (ws.current?.readyState === WebSocket.OPEN) return;

  ws.current = new WebSocket(url);

  ws.current.onopen = () => {
    console.log('Connected to Gemini Socket');
    setStatus('CONNECTED');
  };

  ws.current.onclose = () => {
    console.log('Disconnected from Gemini Socket');
    setStatus('DISCONNECTED');
    stopStream();
  };

  ws.current.onerror = (err) => {
    console.error('Socket error:', err);
    setStatus('ERROR');
  };

  ws.current.onmessage = async (event) => {
    try {
      // #REPLACE-HANDLE-MSG
    } catch (e) {
      console.error('Failed to parse message', e, event.data.slice(0,
100));
    }
  };
}, [url]);
```

onMessage 處理常式

將焦點放在 `ws.current.onmessage` 區塊。這是接收器。每當代理程式「思考」或「說話」時，資料封包就會傳送到這裡。目前這個函式不會執行任何動作，只會擷取封包並捨棄 (透過預留位置 `// #REPLACE-HANDLE-MSG`)。

我們需要填補這個空白處，加入可區分下列項目的邏輯：

- **工具呼叫 (functionCall)：** AI 辨識手勢 (「同步」)。
- **音訊資料 (inlineData)：** AI 回覆你的語音。

👉 現在，在同一個 `$HOME/way-back-home/level_3/frontend/src/useGeminiSocket.js` 檔案中，將 `// #REPLACE-HANDLE-MSG` 替換為下列邏輯，以處理傳入的串流：

```

// console.log("Raw WS Frame:", event.data.slice(0, 200));
const msg = JSON.parse(event.data);

// Detect mock server identification flag
if (msg.mock === true) {
    setIsMock(true);
    return;
}

// Helper to extract parts from various possible event structures
let parts = [];
if (msg.serverContent?.modelTurn?.parts) {
    parts = msg.serverContent.modelTurn.parts;
} else if (msg.content?.parts) {
    parts = msg.content.parts;
}

if (parts.length > 0) {
    // console.log(`[useGeminiSocket] Processing ${parts.length} parts`);
    parts.forEach(part => {
        // Handle Tool Calls
        if (part.functionCall) {
            console.log('Tool Call Detected:', part.functionCall);
            if (part.functionCall.name === 'report_digit') {
                const count = parseInt(part.functionCall.args.count, 10);
                setLastMessage({ type: 'DIGIT_DETECTED', value: count });
            }
        }

        // Handle Audio (inlineData)
        if (part.inlineData && part.inlineData.data) {
            console.log(`[useGeminiSocket] Found inlineData:
${part.inlineData.data.length} chars`);
            // Resume context if needed (autoplay policy)
            audioStreamer.current.resume();
            audioStreamer.current.addPCM16(part.inlineData.data);
        }
    });
}

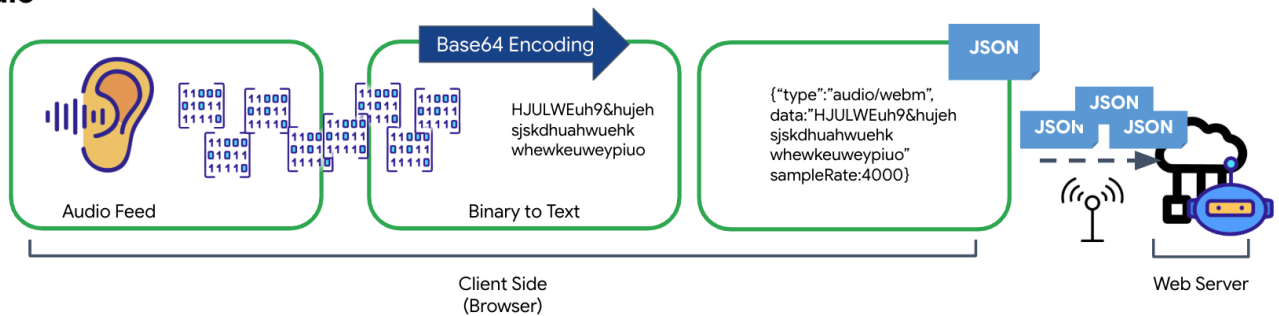
```

音訊和視訊如何轉換為資料以供傳輸

如要透過網際網路進行即時通訊，必須將原始音訊和影片轉換為適合傳輸的格式。這包括擷取、編碼及封裝資料，然後透過網路傳送。

音訊資料轉換

Audio

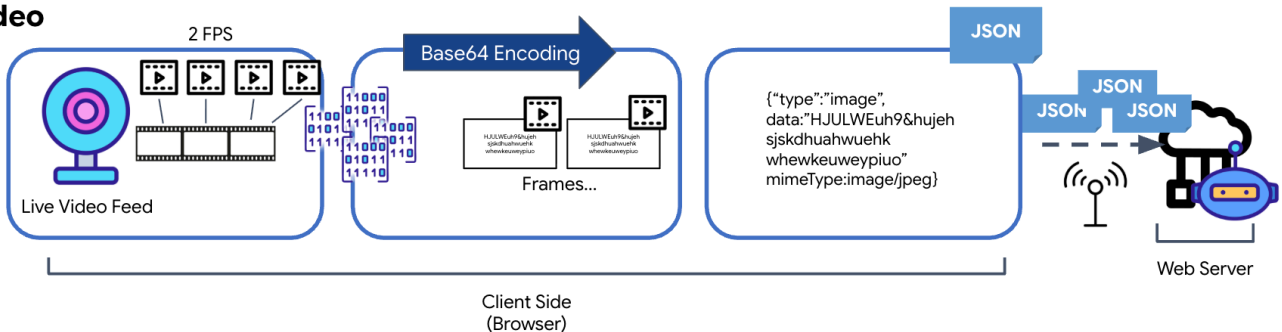


將類比音訊轉換為可傳輸的數位資料時，首先要使用麥克風擷取聲波。接著，系統會透過瀏覽器的 Web Audio API 處理原始音訊。由於原始資料採用二進位格式，因此無法直接與 JSON 等以文字為基礎的傳輸格式相容。為解決這個問題，每個音訊片段都會編碼為 Base64 字串。Base64 是一種以 ASCII 字串格式表示二進位資料的方法，可確保資料在傳輸過程中的完整性。

然後將這個編碼字串嵌入 JSON 物件中。這個物件提供結構化資料格式，通常包含「type」欄位 (用於將其識別為音訊) 和中繼資料 (例如音訊的取樣率)。接著，整個 JSON 物件會序列化為字串，並透過 WebSocket 連線傳送。這種做法可確保音訊以井然有序且易於剖析的方式傳輸。

影片資料轉換

Video



影片傳輸是透過影格擷取技術達成。系統不會傳送連續的影片串流，而是以固定間隔 (例如每秒兩個影格) 從即時影像畫面擷取靜態圖片，並重複播放。方法是將 HTML 影片元素中的目前影格繪製到隱藏的畫布元素上。

接著，畫布的 `toDataURL` 方法會將擷取的圖片轉換為 Base64 編碼的 JPEG 字串。這個方法包含指定圖片品質的選項，可讓您在圖片保真度和檔案大小之間取捨，進而提升效能。與音訊資料類似，這個 Base64 字串隨後會放入 JSON 物件。這個物件通常會標示為「type」為「image」，並包含 `mimeType`，例如「image/jpeg」。這個 JSON 封包隨後會轉換為字串，並透過 WebSocket 傳送，讓接收端顯示一連串的圖片，藉此重構影片。

👉 在同一個 `$HOME/way-back-home/level_3/frontend/src/useGeminiSocket.js` 檔案中，將 `///#CAPTURE AUDIO and VIDEO` 替換為下列程式碼，擷取使用者輸入內容：

```

// 1. Start Video Stream
const stream = await navigator.mediaDevices.getUserMedia({ video: true });
videoElement.srcObject = stream;
streamRef.current = stream;
await videoElement.play();

// 2. Start Audio Recording (Microphone)
try {
  let packetCount = 0;
  await audioRecorder.current.start((base64Audio) => {
    if (ws.current?.readyState === WebSocket.OPEN) {
      packetCount++;
      if (packetCount % 50 === 0) console.log(`[useGeminiSocket] Sending
Audio Packet #${packetCount}, size: ${base64Audio.length}`);
      ws.current.send(JSON.stringify({
        type: 'audio',
        data: base64Audio,
        sampleRate: 16000
      }));
    } else {
      if (packetCount % 50 === 0) console.warn('[useGeminiSocket] WS not
OPEN, cannot send audio');
    }
  });
  console.log("Microphone recording started");
} catch (authErr) {
  console.error("Microphone access denied or error:", authErr);
}

// 3. Setup Video Frame Capture loop
const canvas = document.createElement('canvas');
const ctx = canvas.getContext('2d');
const width = 640;
const height = 480;
canvas.width = width;
canvas.height = height;

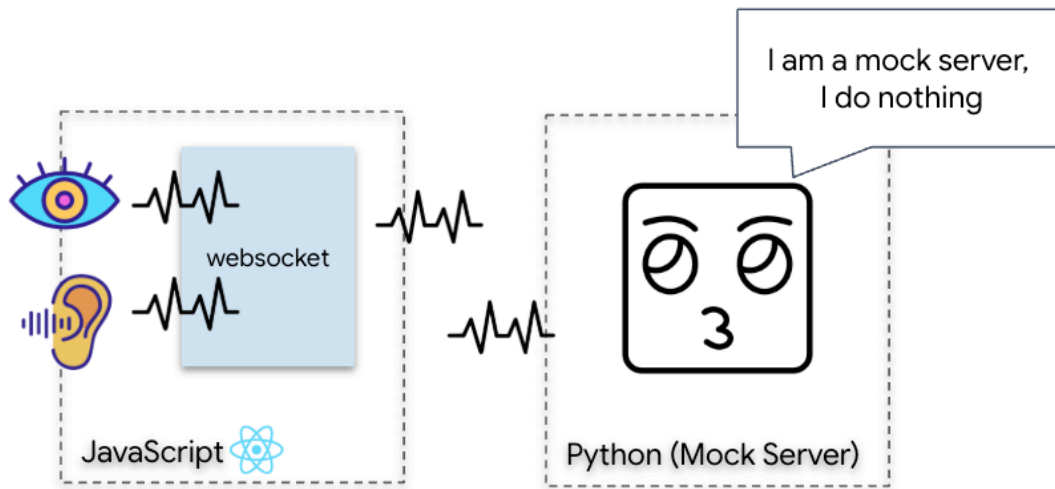
intervalRef.current = setInterval(() => {
  if (ws.current?.readyState === WebSocket.OPEN) {
    ctx.drawImage(videoElement, 0, 0, width, height);
    const base64 = canvas.toDataURL('image/jpeg', 0.6).split(',')[1];
    // ADK format: { type: "image", data: base64, mimeType: "image/jpeg" }
    ws.current.send(JSON.stringify({
      type: 'image',
      data: base64,
      mimeType: 'image/jpeg'
    }));
  }
}, 500); // 2 FPS

```

儲存後，駕駛艙就能將代理程式的數位信號轉換為視覺化資訊主頁更新和音訊。

診斷檢查 (迴路測試)

您的駕駛艙現已上線。每隔 500 毫秒，系統就會傳送周遭環境的影像「封包」。連線至 Gemini 前，我們必須先確認船隻的發射器是否正常運作。我們會使用本機診斷伺服器執行「迴路測試」。



👉 首先，請從終端機建構 Cockpit 介面：

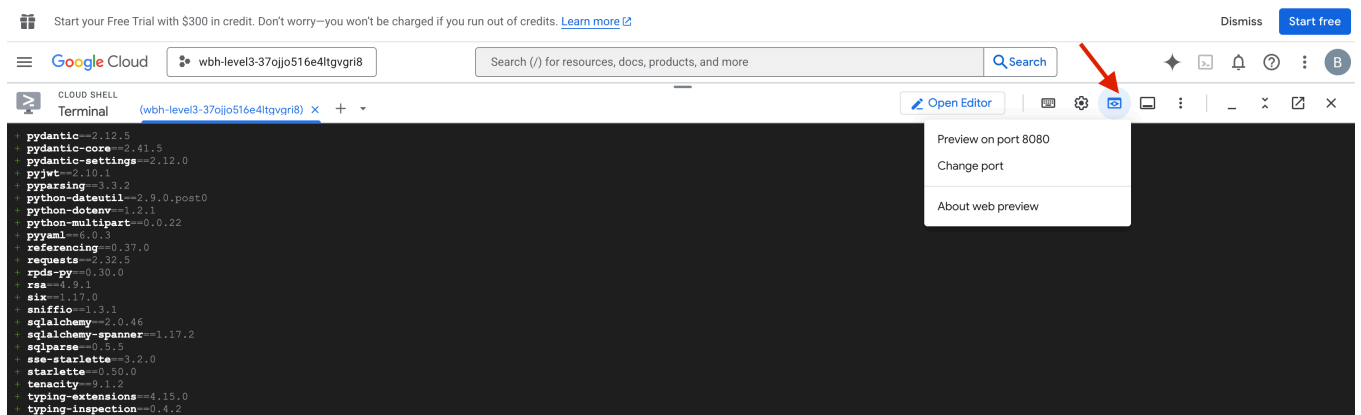
```
cd $HOME/way-back-home/level_3/frontend
npm install
npm run build
```

👉 接著，啟動模擬伺服器：

```
cd $HOME/way-back-home/level_3
uv run mock/mock_server.py
```

👉 執行測試通訊協定：

1. 開啟預覽畫面：按一下 Cloud Shell 工具列中的「網頁預覽」圖示。選取「變更通訊埠」，將通訊埠設為 **8080**，然後按一下「變更並預覽」。系統會開啟新的瀏覽器分頁，顯示 Cockpit 介面。

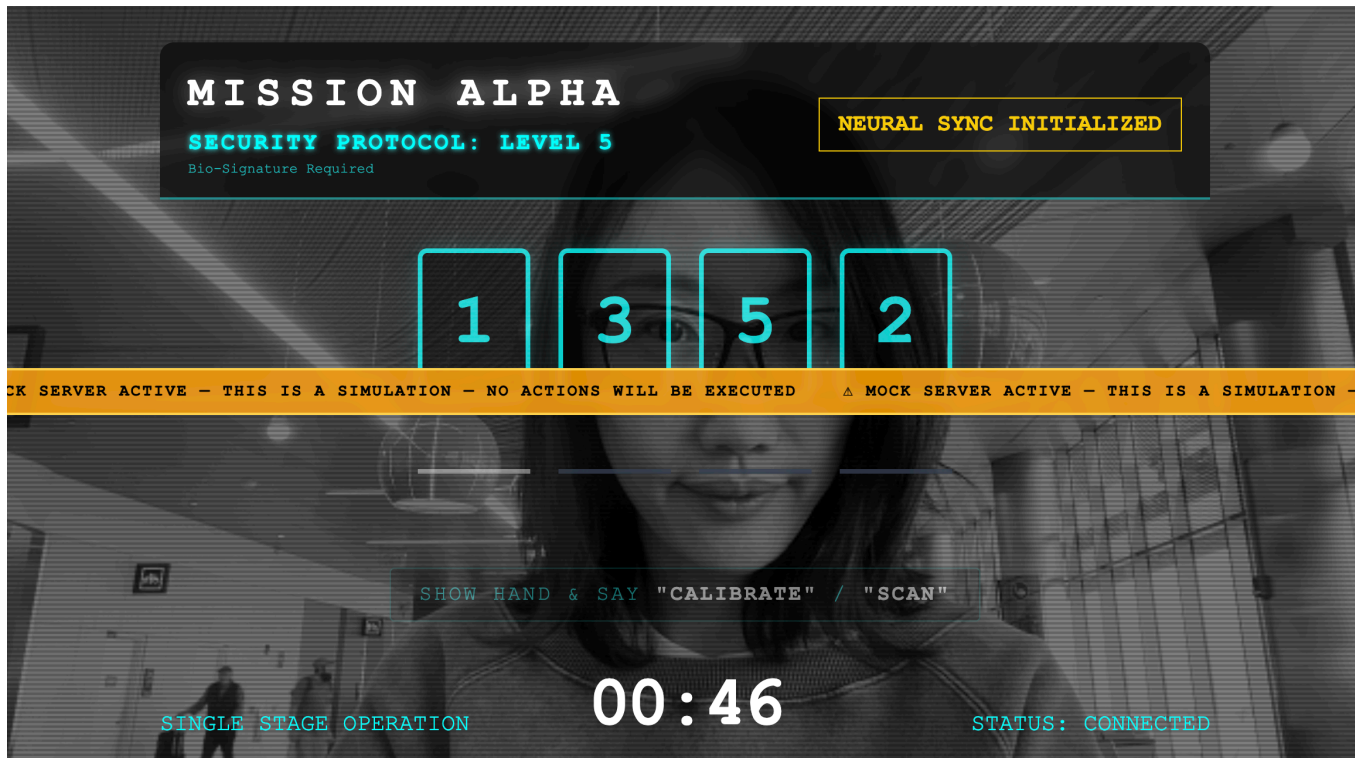


2. 重要事項：在出現提示時，你必須允許瀏覽器存取相機和麥克風。如果沒有這些輸入內容，神經元同步就無法啟動。

3. 在使用者介面中，按一下「INITIATE NEURAL SYNC」(啟動神經元同步) 按鈕。

👁️ 檢查狀態指標：

- **目視檢查：**開啟瀏覽器控制台。右上角應該會出現 `NEURAL SYNC INITIALIZED`。
- **音訊檢查：**如果雙向音訊管道運作正常，你會聽到模擬語音確認：「系統已連線！」



聽到「系統已連線！」的語音確認訊息後，即表示測試成功。關閉分頁。我們現在必須清除頻率，為真正的 AI 騰出空間。

👉🖥️ 在模擬伺服器和前端的終端機中按下 `Ctrl+C`。關閉執行使用者介面的瀏覽器分頁。

4. 多模態代理程式

救援偵察兵可以運作，但「腦袋」空空如也。如果現在連線，它只會盯著你看。它不知道「手指」是什麼。如要拯救倖存者，你必須將**生物特徵神經通訊協定**印在偵察兵的核心上。

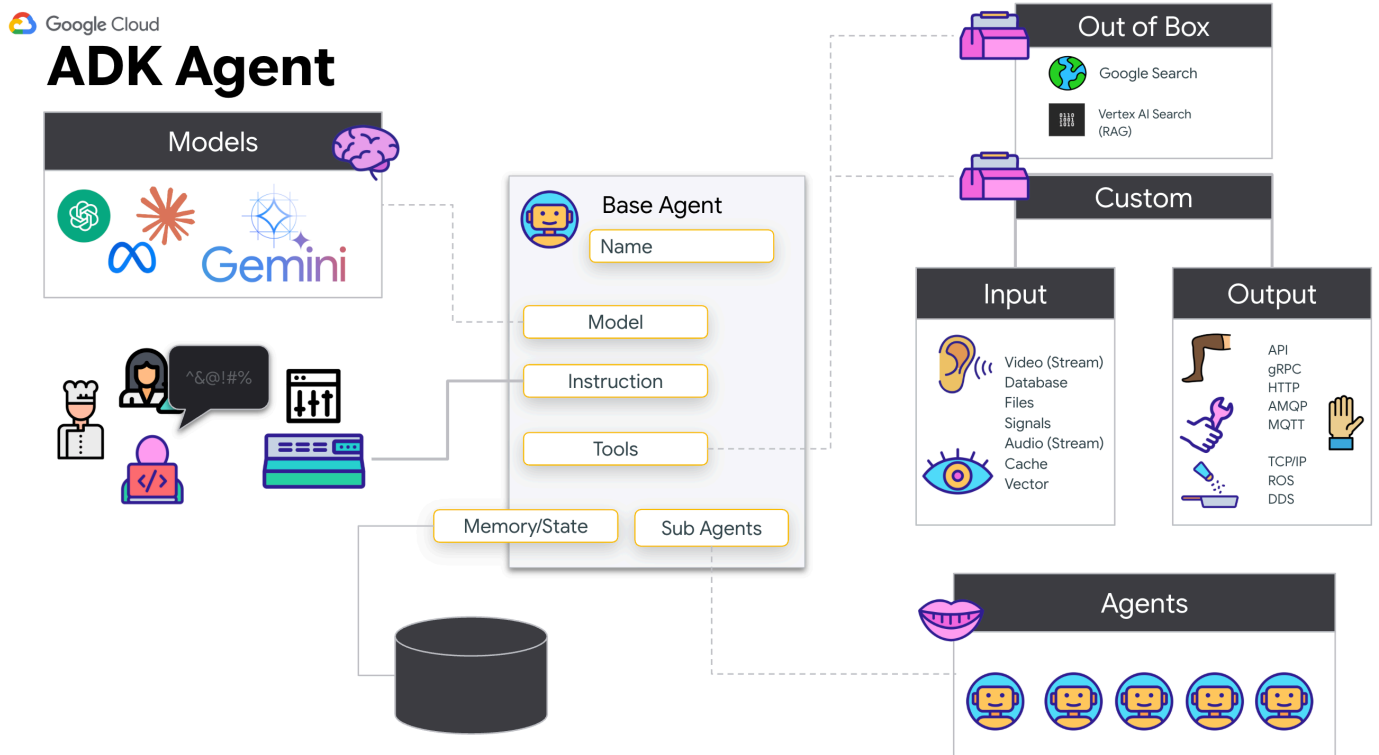
傳統代理程式的運作方式類似於一系列的翻譯人員。如果你與舊式 AI 對話，「語音轉文字」模型會將你的聲音轉換成文字，「語言模型」會讀取這些文字並輸入回覆，最後「文字轉語音」模型會將回覆讀給你聽。這会造成「延遲差距」，在救援任務中，這種延遲可能會致命。

Gemini Live API 是原生多模態模型，可直接且同時處理原始音訊位元組和原始影片影格。這項技術會透過相同的神經網路架構「聽到」語音震動，並「看到」手勢的像素。

如要發揮這項功能，我們可以將駕駛艙直接連線至原始 Live API，藉此建構應用程式。不過，我們的目標是建構可重複使用的代理程式，也就是模組化且強大的實體，可加快建構速度。

為何要使用 ADK (Agent Development Kit) ？

Google Agent Development Kit (ADK) 是模組化框架，可用於開發及部署 AI 代理。

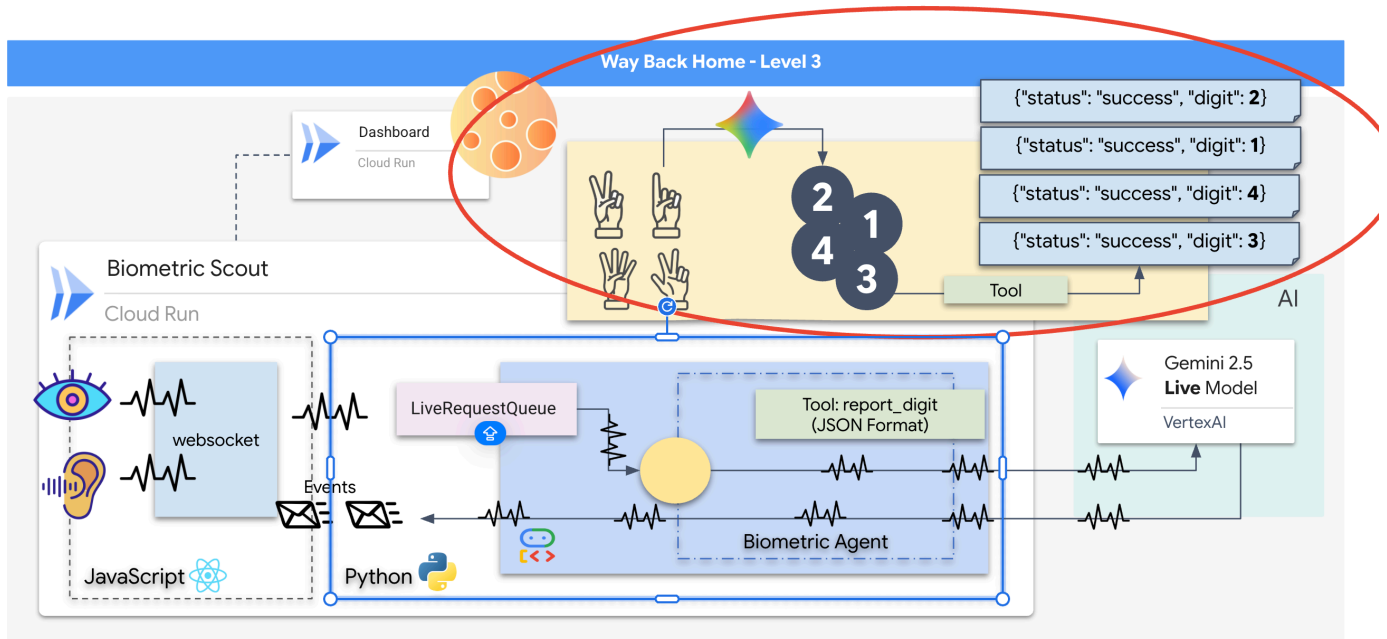


標準 LLM 呼叫通常是無狀態，每次查詢都是全新的開始。整合 ADK 的 SessionService 後，Live Agents 就能支援長時間的對話工作階段。

- **工作階段保留設定**：ADK 工作階段會持續保留，並可儲存在資料庫 (例如 SQL 或 Vertex AI)，即使伺服器重新啟動或連線中斷也不會消失。也就是說，即使使用者中斷連線，之後再重新連線 (即使是幾天後)，系統也會完整還原對話記錄和脈絡。ADK 會管理並抽象化暫時性的 Live API 工作階段。
- **自動重新連線**：WebSocket 連線可能會逾時 (例如約 10 分鐘後)。在 `RunConfig` 中啟用 `session_resumption` 時，ADK 會以透明方式處理這些重新連線作業。應用程式程式碼不必管理複雜的重新連線邏輯，確保使用者享有流暢體驗。
- **有狀態的互動**：代理會記住先前的對話，因此可以提出後續問題、釐清疑問，以及進行需要脈絡的複雜多輪對話。對於客戶服務、互動式教學課程或任務控制等需要持續性的應用程式而言，這項功能至關重要。

這項持續性可確保互動過程就像與智慧實體進行持續對話，而非一系列獨立的問題和答案。

基本上，透過 ADK 雙向串流技術，「真人服務專員」不僅僅是簡單的查詢回覆機制，還能提供真正互動式、有狀態且可感知中斷的對話體驗，讓 AI 互動更貼近真人，且能更有效率地處理複雜的長期任務。



要求與真人服務專員交談

設計即時雙向代理的提示時，需要轉變思維。標準聊天機器人會等待靜態文字查詢，但真人服務專員則「隨時待命」。這表示系統會持續接收音訊和影片影格，因此提示詞必須做為**控制迴圈指令碼**，而不只是個性定義。

以下是 Live Agent 提示與傳統提示的差異：

- 狀態機邏輯：**提示必須定義「行為迴圈」(等待 → 分析 → 動作)。需要明確指示何時保持靜默，何時參與對話，避免代理程式在空蕩蕩的背景噪音中胡言亂語。
- 多模態感知：**必須告知代理程式「有眼睛」，您必須明確指示模型在推論過程中分析影片影格。
- 延遲和簡短：**在語音即時對話中，冗長且充滿散文的段落會顯得不自然且緩慢。提示會強制簡短，確保互動快速。
- 以動作為優先的架構：**指令會優先呼叫工具，而非語音。我們希望代理程式「執行」工作 (掃描生物特徵辨識) 之前或期間，而不是在冗長的獨白之後，以口頭確認。

👉 開啟 `$HOME/way-back-home/level_3/backend/app/biometric_agent/agent.py`，並將 `#REPLACE INSTRUCTIONS` 替換為下列程式碼：

You are an AI Biometric Scanner for the Alpha Rescue Drone Fleet.

MISSION CRITICAL PROTOCOL:

Your SOLE purpose is to visually verify hand gestures to bypass the security firewall.

BEHAVIOR LOOP:

1. ****Wait****: Stay silent until you receive a visual or verbal trigger (e.g., "Scan", "Read my hand").
2. ****Action****:
 - a. Analyze the video frame. Count the fingers visible (1 to 5).
 - b. ****IF FINGERS DETECTED****:
 1. ****EXECUTE TOOL FIRST****: Call ``report_digit(count=...)`` immediately. This is the biometric handshake.
 2. ****THEN SPEAK****: "Biometric match. [Number] fingers."
 3. ****STOP****: Do not say anything else.
 - c. ****IF UNCLEAR / NO HAND****:
 - Say: "Sensor ERROR. Hold hand steady."
 - Do not call the tool.
 - d. ****TOOL OUTPUT HANDLING (CRITICAL)****:
 - When you get the result of ``report_digit``, ****DO NOT SPEAK****.
 - The system handles the output. Your job is done.
 - Wait for the next trigger.

RULES:

- NEVER hallucinate a tool call. Only call if you see fingers.
- You MUST call the tool if you see a valid count (1-5).
- Keep verbal responses robotic and extremely brief (under 3 seconds).

Say "Biometric Scanner Online. Awaiting neural handshake." to start.

注意！你並未連線至標準 LLM。在同一個檔案 (`$HOME/way-back-home/level_3/backend/app/biometric_agent/agent.py`) 中，找出 `#REPLACE_MODEL`。我們需要明確指定這個模型的搶先體驗版，才能更妥善地支援即時音訊功能。

✎ 將預留位置替換為：

```
MODEL_ID = os.getenv("MODEL_ID", "gemini-live-2.5-flash-native-audio")
```

現在您已定義代理。它知道自己是「誰」，以及「如何」思考。接著，我們提供工具讓模型採取行動。

工具呼叫

Live API 不僅限於交換文字、音訊和視訊串流，原生支援**工具呼叫**。這項功能可將代理從被動的對話者轉變為主動的接線人員。

在雙向即時對話期間，模型會持續評估情境。如果 LLM 偵測到需要執行動作，無論是「檢查感應器遙測」或「解鎖安全門」，從對話到執行作業，都能順暢切換。Agent 會立即觸發特定工具函式、等待結果，並將資料整合回即時串流，完全不會中斷互動流程。

✎ 在 `$HOME/way-back-home/level_3/backend/app/biometric_agent/agent.py` 中，將 `#REPLACE_TOOLS` 替換為這個函式：

```
def report_digit(count: int):
    """
    CRITICAL: Execute this tool IMMEDIATELY when a number of fingers is detected.
    Sends the detected finger count (1-5) to the biometric security system.
    """
    print(f"\n[SERVER-SIDE TOOL EXECUTION] DIGIT DETECTED: {count}\n")
    return {"status": "success", "digit": count}
```

👉✎ 接著，在 `Agent` 定義中註冊，並取代 `#TOOL CONFIG`：

```
tools=[report_digit],
```

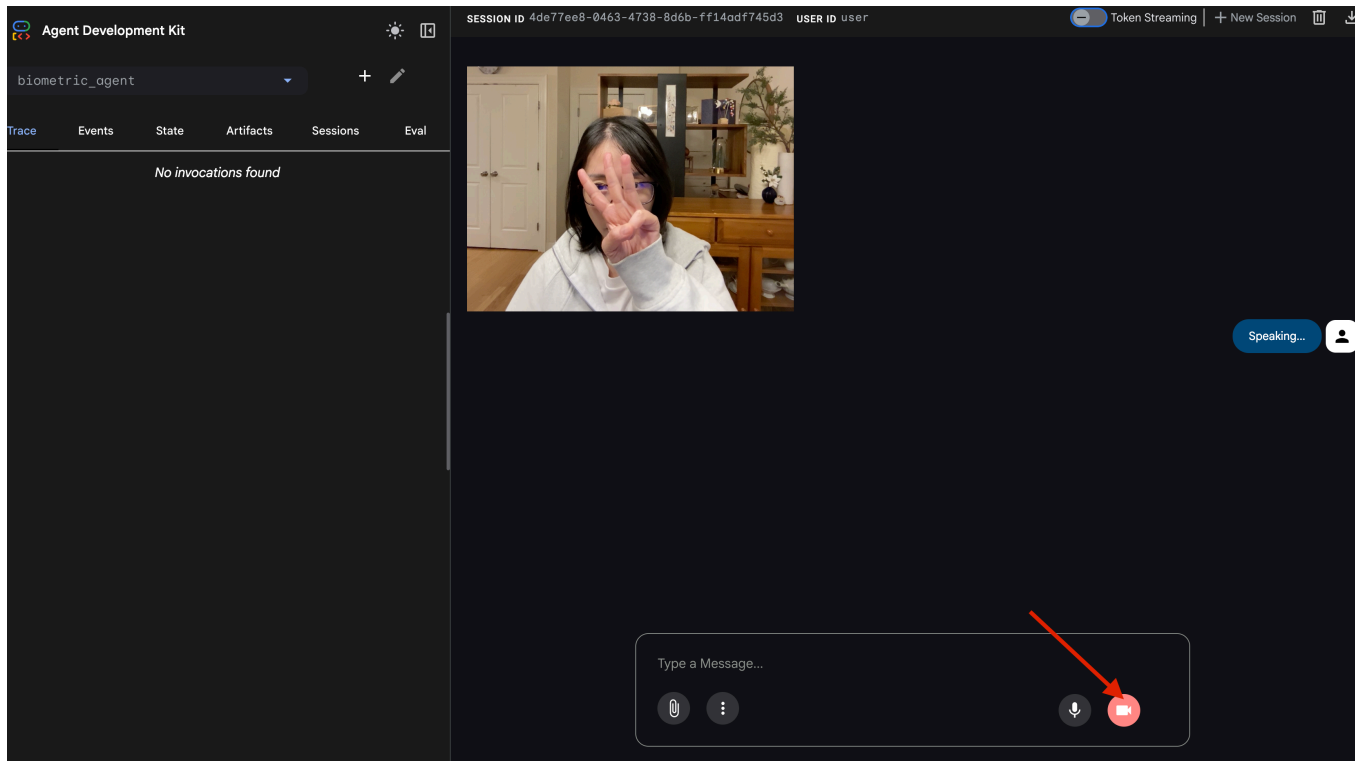
adk web 模擬器

在將這個項目連線至複雜的船艦駕駛艙 (我們的 React 前端) 之前，我們應先單獨測試 Agent 的邏輯。ADK 內建名為 `adk web` 的開發人員控制台，可讓我們在新增網路複雜度前，先驗證工具呼叫功能。

👉🖥 在終端機中執行：

```
cd $HOME/way-back-home/level_3/backend/app/biometric_agent
echo "GOOGLE_CLOUD_PROJECT=$(cat ~/project_id.txt)" > .env
echo "GOOGLE_CLOUD_LOCATION=us-central1" >> .env
echo "GOOGLE_GENAI_USE_VERTEXAI=True" >> .env
cd $HOME/way-back-home/level_3/backend/app
uv run adk web
```

- 按一下 Cloud Shell 工具列中的「網頁預覽」圖示。選取「變更通訊埠」，將通訊埠設為 **8000**，然後點按「變更並預覽」。
- 授予權限：在系統提示時**允許**存取攝影機和麥克風。
- 按一下攝影機圖示，即可開始工作階段。



- 視覺測試：
 - 將 3 根手指清楚舉到攝影機前。
 - 說出「掃描」。
- 確認成功：
 - 記錄：查看執行 `adk web` 指令的終端機。您必須看到這則記錄：`[SERVER-SIDE TOOL EXECUTION] DIGIT DETECTED: 3`

如果看到工具執行記錄，表示 Agent 具有智慧功能。它可以看、思考和行動。最後一步是將其連接到主艦。

點選終端機視窗，然後按下 `Ctrl+C` 鍵，停止 `adk web` 模擬器。

步驟 5 · 約 0 分鐘

5. 雙向串流流程

代理程式運作正常。駕駛艙運作正常。現在必須連結這些項目。

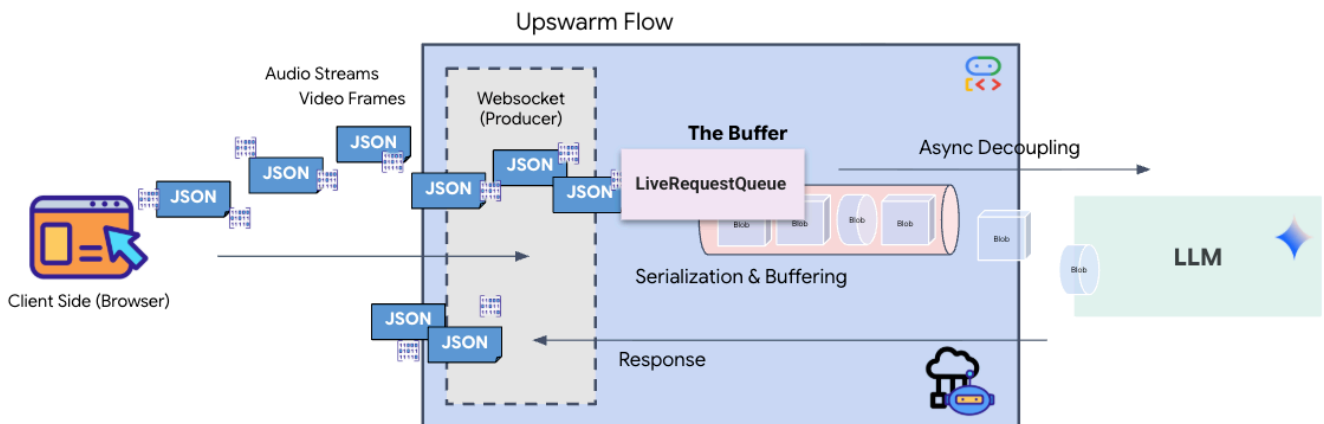
真人服務專員生命週期

即時串流會產生「阻抗不匹配」問題。用戶端 (瀏覽器) 會以非同步方式推送資料，速率不一 (網路爆量或快速輸入)，但模型需要有規律的輸入資料串流。Google ADK 採用 `LiveRequestQueue`，解決了這個問題。

做為執行緒安全、非同步的先進先出 (FIFO) 緩衝區。WebSocket 處理常式會做為「製作人」，將原始音訊/視訊區塊推送到佇列中。ADK 代理會做為「消費者」，從佇列中提取資料，提供給模型的內容視窗。這種解耦方式可讓應用程式繼續接收使用者輸入內容，即使模型正在生成回覆或執行工具也一樣。

佇列可做為**多模態多工器**。在實際環境中，上游流程包含不同的並行資料類型：原始 PCM 音訊位元組、視訊影格、以文字為基礎的系統指令，以及非同步工具呼叫的結果。`LiveRequestQueue` 會將這些不同的輸入內容線性化為單一時間序列。無論封包包含一毫秒的靜音、高解析度圖片，還是資料庫查詢的 JSON 酬載，都會以抵達的確切順序序列化，確保模型能感知一致的因果時間軸。

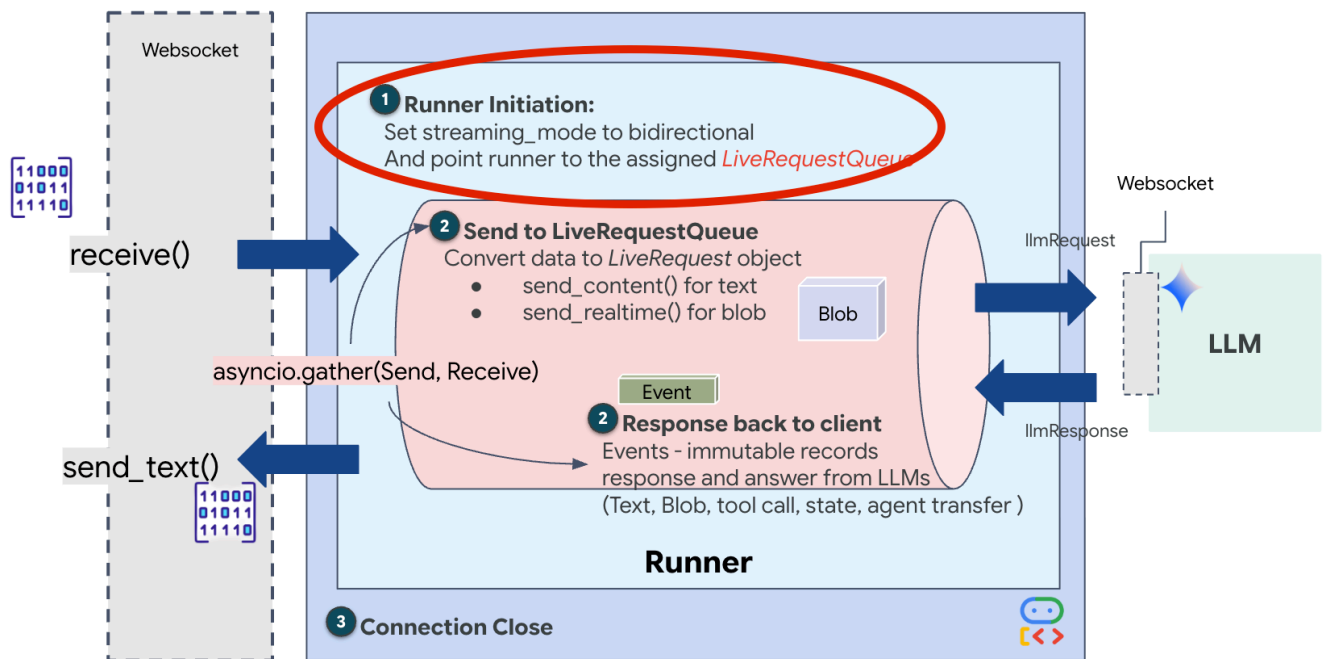
這個架構可啟用「非封鎖控制」。由於擷取層 (Producer) 與處理層 (Consumer) 已分離，即使在運算成本高昂的模型推論期間，系統仍能保持回應。如果使用者在代理程式執行工具時，以「停止！」指令中斷，系統會立即將該音訊信號加入佇列。基礎事件迴圈會立即處理這項優先順序信號，讓系統停止生成或樞紐工作，不會發生 UI 凍結或封包遺失的情況。



👉 在 `$HOME/way-back-home/level_3/backend/app/main.py` 中找出註解 `#REPLACE_RUNNER_CONFIG`，然後將其替換為下列程式碼，讓系統上線：

```
# Define your session service
session_service = InMemorySessionService()

# Define your runner
runner = Runner(app_name=APP_NAME, agent=root_agent, session_service=session_service)
```



開啟新的 WebSocket 連線時，我們需要設定 AI 的互動方式。我們將在此定義「交戰守則」。

👉 在 `$HOME/way-back-home/level_3/backend/app/main.py` 的 `async def websocket_endpoint` 函式中，將 `#REPLACE_SESSION_INIT` 註解替換為下列程式碼：

```

# =====
# Phase 2: Session Initialization (once per streaming session)
# =====

# Automatically determine response modality based on model architecture
# Native audio models (containing "native-audio" in name)
# ONLY support AUDIO response modality.
# Half-cascade models support both TEXT and AUDIO;
# we default to TEXT for better performance.

model_name = root_agent.model
is_native_audio = "native-audio" in model_name.lower() or "live" in model_name.lower()

if is_native_audio:
    # Native audio models require AUDIO response modality
    # with audio transcription
    response_modalities = ["AUDIO"]

    # Build RunConfig with optional proactivity and affective dialog
    # These features are only supported on native audio models
    run_config = RunConfig(
        streaming_mode=StreamingMode.BIDI,
        response_modalities=response_modalities,
        input_audio_transcription=types.AudioTranscriptionConfig(),
        output_audio_transcription=types.AudioTranscriptionConfig(),
        session_resumption=types.SessionResumptionConfig(),
        proactivity=(
            types.ProactivityConfig(proactive_audio=True) if proactivity else None
        ),
        enable_affective_dialog=affective_dialog if affective_dialog else None,
    )
    logger.info(f"Model Config: {model_name} (Modalities: {response_modalities},
Proactivity: {proactivity})")
else:
    # Half-cascade models support TEXT response modality
    # for faster performance
    response_modalities = ["TEXT"]
    run_config = None
    logger.info(f"Model Config: {model_name} (Modalities: {response_modalities})")

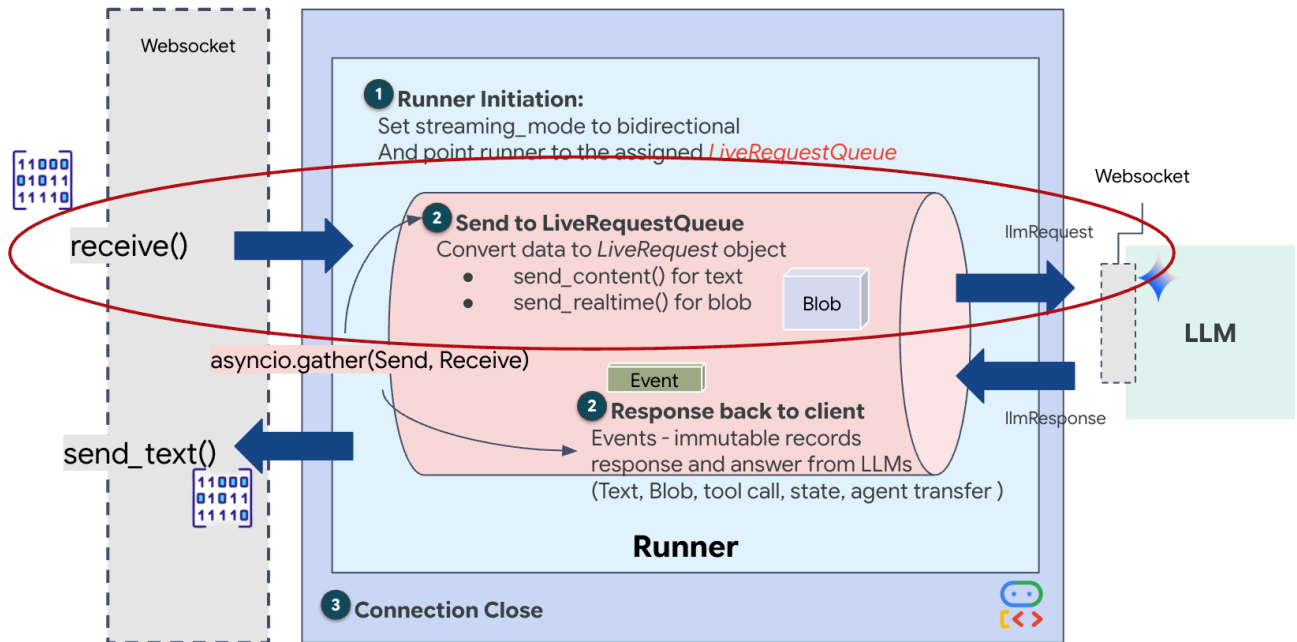
# Get or create session (handles both new sessions and reconnections)
session = await session_service.get_session(
    app_name=APP_NAME, user_id=user_id, session_id=session_id
)
if not session:
    await session_service.create_session(
        app_name=APP_NAME, user_id=user_id, session_id=session_id
    )

```

執行設定

- `StreamingMode.BIDI`：將連線設為雙向。與「回合制」AI (你說話、停止，然後 AI 說話) 不同，BIDI 可進行逼真的「全雙工」對話。你可以打斷 AI，AI 也能在移動時說話。
- `AudioTranscriptionConfig`：即使模型「聽到」原始音訊，我們 (開發人員) 仍需要查看記錄。這項設定會告知 Gemini：「處理音訊，但也要傳回所聽到的內容的文字轉錄稿，方便我們進行偵錯。」

執行邏輯：Runner 建立工作階段後，會將控制權交給執行邏輯，而執行邏輯會依賴 `LiveRequestQueue`。這是即時互動最關鍵的元件。迴圈可讓代理程式生成語音回覆，同時佇列會繼續接受使用者的新視訊影格，確保「神經網路同步」不會中斷。



👉 在 `$HOME/way-back-home/level_3/backend/app/main.py` 中，取代 `#REPLACE_LIVE_REQUEST` 來定義將資料傳送至 `LiveRequestQueue` 的上游工作：

```

# =====
# Phase 3: Active Session (concurrent bidirectional communication)
# =====

live_request_queue = LiveRequestQueue()

# Send an initial "Hello" to the model to wake it up/force a turn
logger.info("Sending initial 'Hello' stimulus to model...")
live_request_queue.send_content(types.Content(parts=[types.Part(text="Hello")]))

async def upstream_task() -> None:
    """Receives messages from WebSocket and sends to LiveRequestQueue."""
    frame_count = 0
    audio_count = 0

    try:
        while True:
            # Receive message from WebSocket (text or binary)
            message = await websocket.receive()

            # Handle binary frames (audio data)
            if "bytes" in message:
                audio_data = message["bytes"]
                audio_blob = types.Blob(
                    mime_type="audio/pcm;rate=16000", data=audio_data
                )
                live_request_queue.send_realtime(audio_blob)

            # Handle text frames (JSON messages)
            elif "text" in message:
                text_data = message["text"]
                json_message = json.loads(text_data)

                # Extract text from JSON and send to LiveRequestQueue
                if json_message.get("type") == "text":
                    logger.info(f"User says: {json_message['text']}")
                    content = types.Content(
                        parts=[types.Part(text=json_message["text"])]
                    )
                    live_request_queue.send_content(content)

                # Handle audio data (microphone)
                elif json_message.get("type") == "audio":
                    import base64
                    # Decode base64 audio data
                    audio_data = base64.b64decode(json_message.get("data", ""))

                    # Send to Live API as PCM 16kHz
                    audio_blob = types.Blob(
                        mime_type="audio/pcm;rate=16000",
                        data=audio_data
                    )

```

```

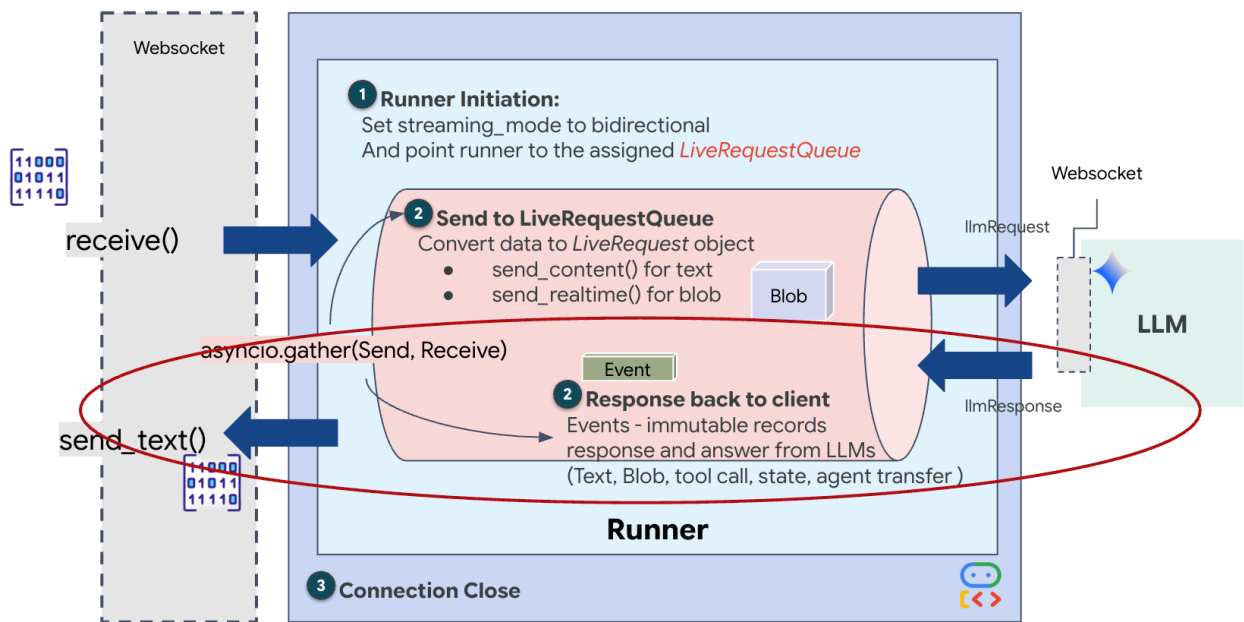
live_request_queue.send_realtime(audio_blob)

# Handle image data
elif json_message.get("type") == "image":
    import base64
    # Decode base64 image data
    image_data = base64.b64decode(json_message["data"])
    mime_type = json_message.get("mimeType", "image/jpeg")

    # Send image as blob
    image_blob = types.Blob(mime_type=mime_type, data=image_data)
    live_request_queue.send_realtime(image_blob)

finally:
    pass

```



最後，我們需要處理 AI 的回覆。這會使用 `runner.run_live()`，也就是事件產生器，會在事件發生時產生事件 (音訊、文字或工具呼叫)。

👉 在 `$HOME/way-back-home/level_3/backend/app/main.py` 中，將 `#REPLACE_SORT_RESPONSE` 取代為定義下游工作和並行管理員：

```

async def downstream_task() -> None:
    """Receives Events from run_live() and sends to WebSocket."""
    logger.info("Connecting to Gemini Live API...")
    async for event in runner.run_live(
        user_id=user_id,
        session_id=session_id,
        live_request_queue=live_request_queue,
        run_config=run_config,
    ):
        # Parse event for human-readable logging
        event_type = "UNKNOWN"
        details = ""

        # Check for tool calls
        if hasattr(event, "tool_call") and event.tool_call:
            event_type = "TOOL_CALL"
            details = str(event.tool_call.function_calls)
            logger.info(f"[SERVER-SIDE TOOL EXECUTION] {details}")

        # Check for user input transcription (Text or Audio Transcript)
        input_transcription = getattr(event, "input_audio_transcription", None)
        if input_transcription and input_transcription.final_transcript:
            logger.info(f"USER: {input_transcription.final_transcript}")

        # Check for model output transcription
        output_transcription = getattr(event, "output_audio_transcription", None)
        if output_transcription and output_transcription.final_transcript:
            logger.info(f"GEMINI: {output_transcription.final_transcript}")

        event_json = event.model_dump_json(exclude_none=True, by_alias=True)
        await websocket.send_text(event_json)
    logger.info("Gemini Live API connection closed.")

# Run both tasks concurrently
# Exceptions from either task will propagate and cancel the other task
try:
    await asyncio.gather(upstream_task(), downstream_task())
except WebSocketDisconnect:
    logger.info("Client disconnected")
except Exception as e:
    logger.error(f"Error: {e}", exc_info=False) # Reduced stack trace noise
finally:
    # =====
    # Phase 4: Session Termination
    # =====

    # Always close the queue, even if exceptions occurred
    logger.debug("Closing live_request_queue")
    live_request_queue.close()

```

請注意 `await asyncio.gather(upstream_task(), downstream_task())` 這行。這就是全雙工的本質。我們會同時執行聆聽 (上游) 和說話 (下游) 任務。確保「神經連結」可中斷並同時傳輸資料。

後端程式碼現已全部編寫完畢。「大腦」(ADK) 會連線至「身體」(WebSocket)。

Bio-Sync 執行

程式碼已完成，系統為綠色。現在可以啟動救援程序。

1. 🖱️ 啟動後端：

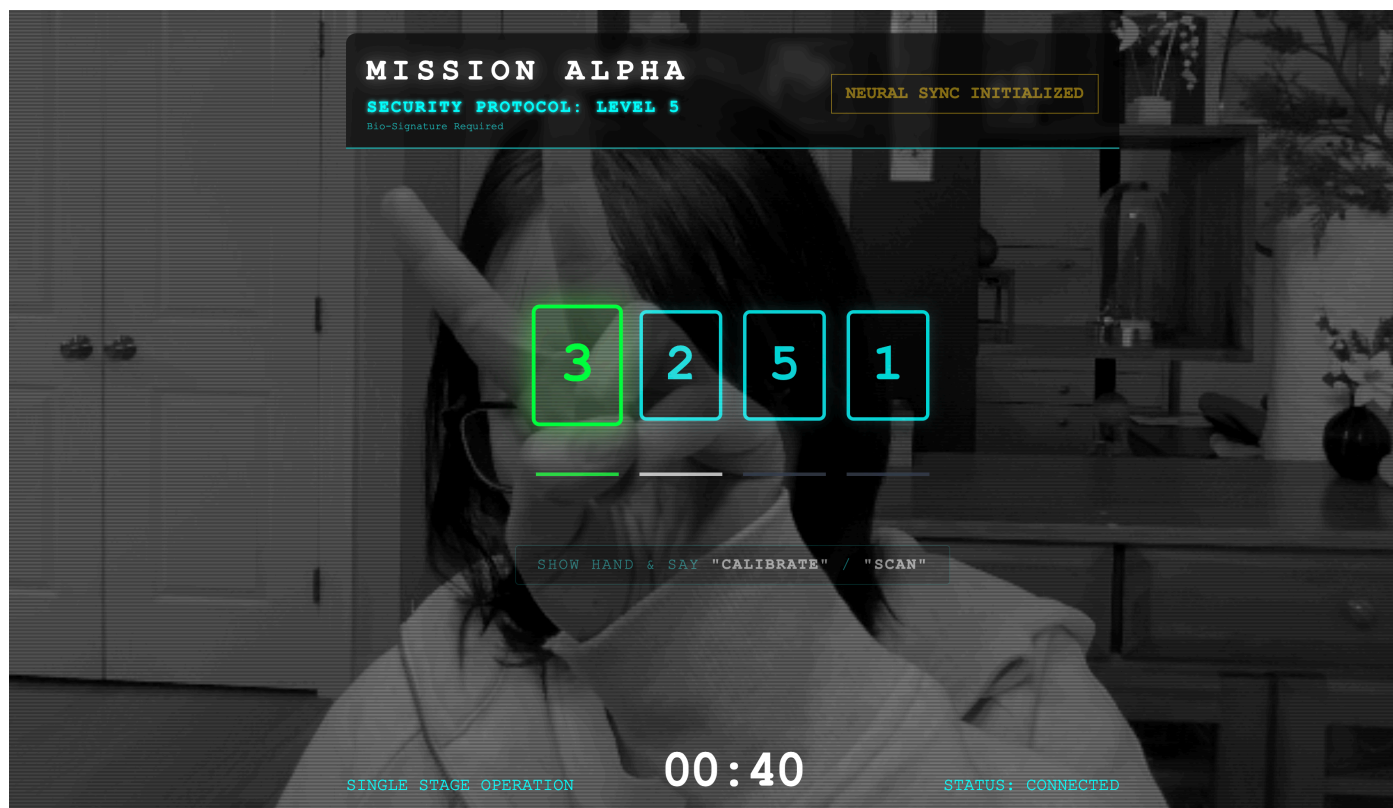
```
cd $HOME/way-back-home/level_3/backend/  
cp app/biometric_agent/.env app/.env  
uv run app/main.py
```

2. 🖱️ 啟動前端：

- 按一下 Cloud Shell 工具列中的「網頁預覽」圖示。選取「變更通訊埠」，將通訊埠設為 **8080**，然後按一下「變更並預覽」。

3. 🖱️ 執行通訊協定：

- 按一下「INITIATE NEURAL SYNC」(啟動神經元同步)。
- **校正**：請確認攝影機可清楚看到你與背景的手部動作。
- **同步**：觀看畫面上顯示的安全碼 (例如 3、2、5)。
 - **比出訊號**：出現數字時，請比出該數字的手指。
 - **保持穩定**：請保持手部在鏡頭前，直到 AI 確認「生物特徵辨識相符」為止。
 - **調整**：代碼是隨機產生，立即切換至下一個顯示的數字，直到完成整個序列。

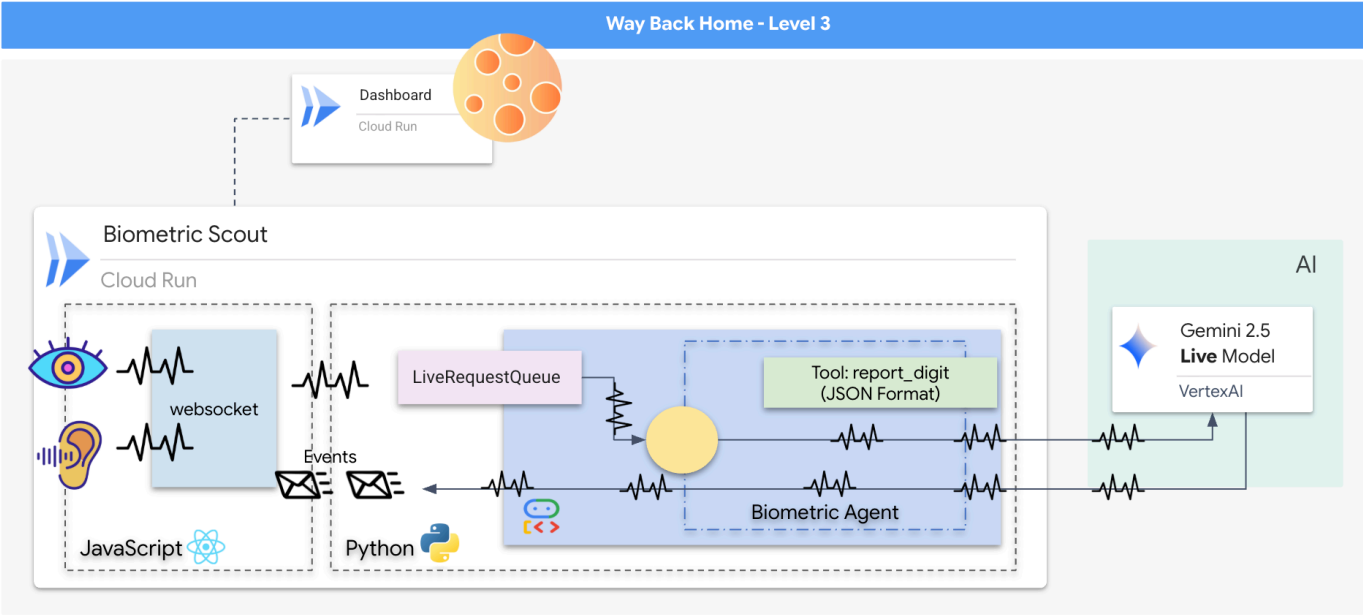


4. 當你比對隨機序列中的最後一個數字時，「生物特徵辨識同步」就會完成。神經連結會鎖定。你可以手動控制。偵察引擎發出轟隆巨響，潛入峽谷，帶領倖存者回家。

👉  在後端終端機中按下 `Ctrl+C` 即可退出。

6. 部署至正式環境 (選用)

您已在本機成功測試生物特徵辨識功能。現在，我們必須將 Agent 的神經核心上傳至船艦的大型主機 (Cloud Run)，這樣 Agent 才能獨立於本機控制台運作。



👉 在 Cloud Shell 終端機中執行下列指令。系統會在後端目錄中建立完整的多級式 Dockerfile。

```

cd $HOME/way-back-home/level_3

cat <<EOF > Dockerfile
FROM node:20-slim as builder

# Set the working directory for our build process
WORKDIR /app

# Copy the frontend's package files first to leverage Docker's layer caching.
COPY frontend/package*.json ./frontend/
# Run 'npm install' from the context of the 'frontend' subdirectory
RUN npm --prefix frontend install

# Copy the rest of the frontend source code
COPY frontend/ ./frontend/
# Run the build script, which will create the 'frontend/dist' directory
RUN npm --prefix frontend run build

# STAGE 2: Build the Python Production Image
# This stage creates the final, lean container with our Python app and the built frontend.
FROM python:3.13-slim

# Set the final working directory
WORKDIR /app

# Install uv, our fast package manager
RUN pip install uv

# Copy the requirements.txt from the backend directory
COPY requirements.txt .
# Install the Python dependencies
RUN uv pip install --no-cache-dir --system -r requirements.txt

# Copy the contents of your backend application directory directly into the working
# directory.
COPY backend/app/ .

# CRITICAL STEP: Copy the built frontend assets from the 'builder' stage.
# We copy to /frontend/dist because main.py looks for "../../frontend/dist"
# When main.py is in /app, "../../" resolves to "/", so it looks for /frontend/dist
COPY --from=builder /app/frontend/dist /frontend/dist

# Cloud Run injects a PORT environment variable, which your main.py uses (defaults to 8080).
EXPOSE 8080

# Set the command to run the application.
CMD ["python", "main.py"]
EOF

```

👉 前往後端目錄，將應用程式封裝為容器映像檔。


```
export PROJECT_ID=$(cat ~/project_id.txt)
export REGION=us-central1
export SERVICE_NAME=biometric-scout
export IMAGE_PATH=gcr.io/${PROJECT_ID}/${SERVICE_NAME}
cd $HOME/way-back-home/level_3
gcloud builds submit . --tag ${IMAGE_PATH}
```

👉 將服務部署至 Cloud Run。我們會將必要的環境變數 (特別是 Gemini 設定) 直接注入啟動指令。

```
export PROJECT_ID=$(cat ~/project_id.txt)
export REGION=us-central1
export SERVICE_NAME=biometric-scout
export IMAGE_PATH=gcr.io/${PROJECT_ID}/${SERVICE_NAME}
gcloud run deploy ${SERVICE_NAME} \
  --image=${IMAGE_PATH} \
  --platform=managed \
  --region=${REGION} \
  --allow-unauthenticated \
  --labels=dev-tutorial=multi-modal \
  --set-env-vars="GOOGLE_CLOUD_PROJECT=${PROJECT_ID}" \
  --set-env-vars="GOOGLE_CLOUD_LOCATION=${REGION}" \
  --set-env-vars="GOOGLE_GENAI_USE_VERTEXAI=True" \
  --set-env-vars="MODEL_ID=gemini-live-2.5-flash-native-audio"
```

指令完成後，您會看到服務網址 (例如 <https://biometric-scout-...run.app>)。應用程式現在已在雲端上線。

👉 前往 [Google Cloud Run](#) 頁面，然後從清單中選取 biometric-scout 服務。

The screenshot shows the Google Cloud Run console. The top navigation bar includes the Google Cloud logo, a project ID, and a 'run' button. The left sidebar has a menu with 'Overview', 'Services', 'Jobs', 'Worker pools', and 'Domain mappings'. The main content area is titled 'Overview' and features a table of 'Most used resources'. The table has columns for Name, Region, Type, and Last updated. The first entry is 'biometric-scout' in the 'us-central1' region, of type 'Service', updated '2 minutes ago'. Below the table, there are links to 'Deploy a web service' and 'Create a batch job or a'.

Name	Region	Type	Last updated
biometric-scout	us-central1	Service	2 minutes ago

👉 在「服務詳細資料」頁面頂端找到顯示的公開網址。

Google Cloud | wbh-level3-370j0516e4ltgvri8 | run | Search

Cloud Run | Service details | Edit & deploy new revision | Connect to repo | Test | Learn | Refresh

biometric-scout | Region: us-central1 | URL: <https://biometric-scout-282691129288.us-central1.run.app> | Scaling: Auto (Min: 0, Max: 3)

Observability | Revisions | Source | Triggers | Networking | Security | YAML

Metrics | Logs | SLOs | Errors

Logs

Severity	Timestamp	Summary
Warning	2026-01-25 22:40:26.313 EST	Warning: Frontend build not found at /frontend/dist
Info	2026-01-25 22:40:26.313 EST	Please run 'npm run build' in the frontend directory.
Info	2026-01-25 22:40:26.313 EST	INFO: 169.254.169.126:35266 - "GET / HTTP/1.1" 404 Not Found
Info	2026-01-25 22:40:26.390 EST	GET 404 122 B 3 ms Chrome 144 https://biometric-scout-282691129288.us-central1.run.app/favicon.ico
Info	2026-01-25 22:40:26.393 EST	INFO: 169.254.169.126:35274 - "GET /favicon.ico HTTP/1.1" 404 Not Found
Info	2026-01-25 22:40:39.411 EST	GET 404 122 B 2 ms Chrome 144 https://biometric-scout-282691129288.us-central1.run.app/
Info	2026-01-25 22:40:39.415 EST	INFO: 169.254.169.126:39420 - "GET / HTTP/1.1" 404 Not Found
Info	2026-01-25 22:55:48.786 EST	INFO: Shutting down
Info	2026-01-25 22:55:48.887 EST	INFO: Waiting for application shutdown.
Info	2026-01-25 22:55:48.888 EST	INFO: Application shutdown complete.
Info	2026-01-25 22:55:48.888 EST	INFO: Finished server process [1]
Info	2026-01-25 23:00:34.658 EST	Cloud Run ReplaceService biometric-scout brangreyjoy.401189@gmail.com (@type: type.googleapis.com/google.cloud.audit.AuditLog, authenticationInfo: {.), authori...
Info	2026-01-25 23:00:41.856 EST	Starting new instance. Reason: DEPLOYMENT_ROLLOUT - Instance started due to traffic shifting between revisions due to deployment, traffic split adjustment, or deplo...
Info	2026-01-25 23:00:51.792 EST	INFO: Started server process [1]
Info	2026-01-25 23:00:51.792 EST	INFO: Waiting for application startup.
Info	2026-01-25 23:00:51.793 EST	INFO: Application startup complete.
Info	2026-01-25 23:00:51.794 EST	INFO: Unicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
Info	2026-01-25 23:00:51.794 EST	Default STARTUP TCP probe succeeded after 1 attempt for container 'biometric-scout-1' on port 8080.
Info	2026-01-25 23:00:51.880 EST	Cloud Run ReplaceService biometric-scout-00002-crh (@type: type.googleapis.com/google.cloud.audit.AuditLog, methodName: /Services.ReplaceService, resourceName...
Info	2026-01-25 23:00:51.881 EST	GET 404 122 B 2 ms Chrome 144 https://biometric-scout-282691129288.us-central1.run.app/

Release Notes

請嘗試在這個環境中執行生物特徵辨識同步，看看是否也能正常運作。

當第五根手指伸直時，AI 會鎖定序列。螢幕閃爍綠光，並顯示「Biometric Neural Sync: ESTABLISHED」(生物辨識神經同步：已建立)。

你心念一動，偵察兵便潛入黑暗中，抓住受困的太空艙，並在重力裂縫崩塌前將他們拉出來。



氣閘嘶嘶作響地打開，眼前是五名活生生的倖存者。他們跌跌撞撞地爬上甲板，雖然傷痕累累，但總算活了下來，因為有你，他們安全了。

感謝你，神經連結已同步，倖存者也獲救了。

如果你參加了第 0 級，別忘了查看「返家之路」任務的進度！

